

Voici une proposition de nouvelle structure de base de données et la logique de conception pour CodeIgniter 4.

1. Nouvelle Structure de Base de Données (Simplifiée)

Cette structure permet de stocker les informations de santé, les régimes avec leur efficacité (variation de poids par durée) et de tracer les achats/objectifs finaux.

Table	Colonnes Clés	Description
utilisateur	id, nom, poids_actuel, taille, solde, est_gold	Stocke le profil pour le calcul de l'IMC.
type_objectif	id, libelle	“Augmenter”, “Réduire”, “Atteindre IMC idéal”.
regime	id, nom, description, duree_unitaire, prix_unitaire, variation_poids	Important : variation_poids est positif pour une prise, négatif pour une perte.
achat_objectif	id, id_utilisateur, id_type_objectif, id_regime, poids_initial, poids_objectif, date_debut, duree_totale, prix_total	Enregistre l'état final lors de l'achat (Etape 5).

2. Logique de Conception par Étape

Étape 1 & 2 : Calcul IMC et Objectifs (Côté Vue/JS)

Dans ton formulaire, utilise la logique suivante pour l'IMC idéal : * **IMC Actuel** : $IMC = \frac{poids}{taille^2}$
 * **Poids Idéal** : On prend souvent un IMC de référence de **22**. Donc $Poids_{ideal} = 22 \times taille^2$.
 * **AJAX** : Quand l'utilisateur clique sur “Atteindre IMC idéal”, tu remplis automatiquement le champ poids_objectif avec cette valeur.

Étape 3 & 4 : Suggestion et Calcul du Prix Total

Pour la requête SQL, voici comment structurer le calcul dans ton modèle pour obtenir le prix et la durée nécessaires pour atteindre la différence de poids souhaitée.

Requête SQL optimisée :

```
SELECT
    r.*,
    ABS(? / r.variation_poids) * r.duree as duree_totale,
    (ABS(? / r.variation_poids) * r.prix_unitaire) as prix_total
FROM regime r
WHERE
    (? > 0 AND r.variation_poids > 0) -- On cherche une prise de poids
```

OR

```
(? < 0 AND r.variation_poids < 0) -- On cherche une perte de poids
```

Les deux premiers ? reçoivent la différence (*poids_objectif* - *poids_actuel*).

En PHP (CodeIgniter 4) :

```
$diff = $poids_objectif - $poids_actuel;
$signe = ($diff > 0) ? "> 0" : "< 0";

// Dans votre modèle
$query = "SELECT *,
          (ABS($diff / variation_poids) * duree_unitaire) as temps_requis,
          (ABS($diff / variation_poids) * prix_unitaire) as prix_final
        FROM regime
        WHERE variation_poids $signe";
```

3. Étape 5 : Le Paiement et l'Enregistrement

Lors de la confirmation, tu dois insérer les données dans la table *achat_objectif*. C'est cette table qui servira de "contrat" entre l'utilisateur et son programme.

Champs à enregistrer : * **duree_totale** : Le résultat du calcul précédent (ex: 45 jours). * **prix_total** : Le prix calculé (en appliquant la remise de 15% si *est_gold* est à 1). * **date_debut** : La date saisie par l'utilisateur.

Ce que cela change :

1. **Indépendance** : Tu n'as plus besoin de la table *regime_detail* qui multipliait les lignes inutilement. Un régime a une efficacité de base (ex: -2kg en 7 jours), et ton code calcule le reste proportionnellement.
 2. **Précision** : Le calcul est exact par rapport au poids que l'utilisateur veut réellement perdre.
 3. **Simplicité** : Le Back Office n'a plus qu'à faire un CRUD simple sur la table *regime*.
-

Architecture du Flux Utilisateur (tsra rah atao formulaire successive)

- **ÉTAPE 1 : Évaluation Santé (Calcul IMC)**
 - **Entrées** : Poids (*kg*) + Taille (*m*).
 - **Traitement** : * $IMC = \frac{poids}{taille^2}$.
* $Poids_Ideal = 22 \times taille^2$.
 - **Sortie** : Affichage de l'IMC actuel et de la cible idéale.
- **ÉTAPE 2 : Définition de l'Objectif (AJAX)**
 - **Choix** : [Prise de poids] | [Perte de poids] | [Atteindre IMC Idéal].
 - **Logique UI** :
 - * Si *Prise/Perte* Saisie manuelle du **Poids Objectif**.
 - * Si *IMC Idéal* Remplissage automatique via le *Poids_Ideal* calculé à l'étape 1.
- **ÉTAPE 3 : Filtrage des Régimes**
 - **Condition** : $\Delta Poids = Poids_Objectif - Poids_Actuel$.

- Logique SQL :
 - * Si $\Delta Poids > 0$ WHERE variation_poids > 0 (Régimes de prise).
 - * Si $\Delta Poids < 0$ WHERE variation_poids < 0 (Régimes de perte).
 - ÉTAPE 4 : Calcul Dynamique (Cœur du Système)
 - Algorithme : Calcul proportionnel basé sur l'efficacité du régime.
 - Variables : * Durée Totale = $ABS(\frac{\Delta Poids}{variation_poids}) \times duree_unitaire$.
 - * Prix Total = $ABS(\frac{\Delta Poids}{variation_poids}) \times prix_unitaire$.
 - Sortie : Liste des régimes avec le prix exact et le temps nécessaire pour atteindre l'objectif précis de l'utilisateur.
 - ÉTAPE 5 : Achat et Suivi (Backend)
 - Paiement : Vérification du solde utilisateur.
 - Remise : Application automatique de **-15%** si l'utilisateur est **Gold**.
 - Persistence : Enregistrement dans une table achat_objectif :
 - * id_regime, poids_initial, poids_objectif, prix_total, duree_totale, date_debut.
-

Points Clés pour les Développeurs

- **Back-Office** : CRUD simple sur la table **regime** (Nom, Prix, Durée, Variation). Plus besoin de gérer des lignes complexes dans **regime_detail**.
- **Front-Office** : Utilisation intensive d'**AJAX** pour que le Poids Objectif et l'IMC réagissent instantanément.
- **Précision** : Le système ne suggère pas juste un régime, il calcule **combien de temps** ce régime spécifique prendra pour atteindre les kilos voulus.